StampFly 勉強会 2026

ドローン制御工学ハンズオン ワークショップ

伊藤 恒平

2026/3/2 – 3/6

資料ダウンロード / Slide Download

Docswell

スライド資料はこちらからダウンロードできます

https://www.docswell.com/s/Kouhei_Ito/KLV77G-2026-03-02-010945

GitHub リポジトリ / GitHub Repository

stampfly_ecosystem

ファームウェア・ツール・資料の全ソースコード

https://github.com/M5Fly-kanazawa/stampfly_ecosystem

Lesson 0

環境セットアップ

Environment Setup

StampFly Workshop

ワークショップカリキュラム / Workshop Curriculum

Day	Lesson	テーマ
Day 1	0–2	環境セットアップ + モータ + コントローラ
Day 2	3–5	LED + IMU + 初フライト
Day 3	6–8	PID 制御 + テレメトリ + モデリング
Day 4	9–11	姿勢推定 + Python SDK + 競技会準備
Day 5 AM		ホバリングタイム競技会

StampFly ハードウェア / Hardware

項目	仕様
MCU	ESP32-S3 (M5Stamp S3)
IMU	BMI270 (6 軸 400 Hz)
気圧	BMP280
ToF	VL53L3CX
質量	37 g（バッテリ含む）
通信	ESP-NOW + WiFi
バッテリ	LiPo 1S 3.7 V

エコシステム全体像 / Ecosystem Overview

`tools/` 開発ツール	sf CLI
`analysis/` データ解析	Jupyter / Python
`simulator/` シミュレータ	Genesis 3D 物理
`control/` 制御設計	PID, MPC, SIL
`firmware/` 組込みコード	ESP32-S3 / ESP-IDF / C++
`protocol/` 通信仕様	SSOT: YAML → コード自動生成

- `firmware/` — 組込みコード
- `control/` — 制御設計
- `simulator/` — 3D シミュレータ
- `analysis/` — データ解析
- `tools/` — sf CLI
- `protocol/` — 通信仕様

開発ツール sf CLI / Development Tool

カテゴリ	コマンド例	説明
ビルド	`sf build` / `sf flash`	ファームウェア開発
診断	`sf doctor` / `sf monitor`	デバッグ
ログ	`sf log wifi` / `sf log viz`	テレメトリ取得・可視化
シミュレータ	`sf sim run`	仮想環境で練習
キャリブレーション	`sf cal gyro/accel/mag`	センサ校正

ワークフロー

`sf lesson switch N` → `sf lesson build` → `sf lesson flash`

Windows 環境構築 (1/2) / Windows Setup

確認コマンド (CMD)	期待される結果	なければ
`git --version`	git version 2.x	`winget install Git.Git`
`python --version`	Python 3.10 以上	`winget install Python.Python.3.12`

インストール手順（CMD で実行）

1. リポジトリをクローン
2. `install.bat` を実行（ESP-IDF + sfcli 自動インストール）
3. `setup_env.bat` で開発環境をアクティベート

```
git clone https://github.com/M5Fly-kanazawa/stampfly_ecosystem
cd stampfly_ecosystem
install.bat
setup_env.bat
```

Windows 環境構築 (2/2) / Driver & Verification

USB シリアルドライバ

CH9102F ドライバをインストール（M5Stack 製品用）

`https://docs.m5stack.com/en/download`

動作確認

`sf doctor` で環境診断 — すべて OK になれば完了

macOS / Linux ユーザー

`install.sh` + `setup_env.sh` を使用してください

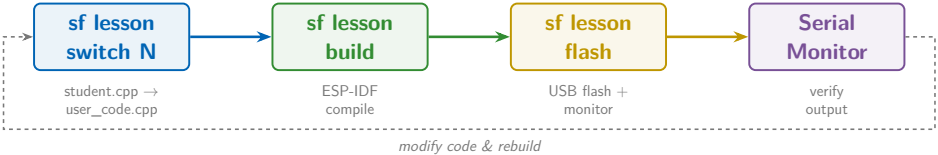
今日のゴール / Today's Goal

目標

ワークショップファームウェアをビルド・書き込み・動作確認する

1. `sf lesson build` でビルド
2. `sf lesson flash` で書き込み
3. シリアルモニタで `"Hello StampFly!"` を確認

開発フロー / Development Flow



sf CLI とは / What is sf CLI?

- StampFly 開発専用のコマンドラインツール
- ESP-IDF のコマンドをシンプルに実行できる

コマンド	説明
`sf doctor`	環境診断
`sf lesson build`	ワークショップビルド
`sf lesson flash`	書き込み＋モニタ
`sf lesson switch N`	レッスン切り替え
`sf monitor`	シリアルモニタ

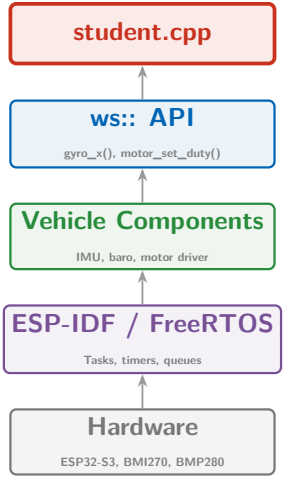
ワークショップの構造 / Workshop Structure

学生が実装する関数は 2 つだけ！

- `setup()` — 起動時に 1 回呼ばれる
- `loop_400Hz(dt)` — 400Hz（2.5ms 毎）で呼ばれる

- ハードウェアの複雑さは `ws::` 名前空間で隠蔽
- FreeRTOS, SPI/I2C, センサーフュージョンを意識する必要なし

ファームウェアの全体像 / Firmware Architecture



← You are here!

setup() / loop_400Hz()

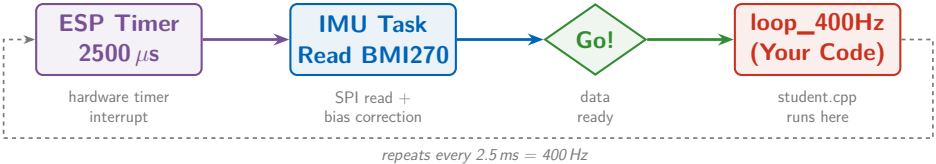
sensor read, filter, motor

xTaskCreate, xQueueSend

SPI, I2C, GPIO

- 皆さんのコードはここ！
  `student.cpp` を編集するだけ
- `ws::gyro_x()` と書くだけで
  SPI 通信→フィルタ→バイアス補正を
  全部やってくれる
- 下のレイヤーはワークショップ中
  意識する必要なし

400Hz ループの裏側 / Behind the 400Hz Loop



ポイント

`loop_400Hz(dt)` は常に IMU データ更新後に呼ばれる— 学生はタイマー管理不要、ただ関数を書くだけ！

実習: Hello StampFly / Hands-on

```cpp
#include "workshop_api.hpp"

void setup()
{
    ws::print("Hello StampFly!");
}

void loop_400Hz(float dt)
{
    // Nothing to do in Lesson 0
}
```

student.cpp

手順

1. `sf lesson switch 0`
2. 上のコードを `student.cpp` に書く
3. `sf lesson build && sf lesson flash`

作業の進め方 / Workflow

ファイル構造

```
firmware/workshop/
  ├── lessons/
  │   ├── lesson_00/student.cpp
  │   ├── lesson_01/student.cpp
  │   └── ...
  └── main/
      └── user_code.cpp ← 編集！
```

1. `sf lesson switch N`
   テンプレートを `user_code.cpp` にコピー
2. `user_code.cpp` を編集
3. `sf lesson build`
4. `sf lesson flash`

注意

`student.cpp` はテンプレートなので直接編集しない！
`switch` で何度でもリセットできる

コントローラのセットアップ / Controller Setup

コントローラのビルドと書き込み

1. コントローラを USB 接続
2. `sf build controller`
3. `sf flash controller`

確認

LCD に起動画面が表示されれば OK
Lesson 2 でペアリングして使用する

シミュレータで遊ぶ / Try the Simulator

USB HID モードに切替

1. 画面タッチでメニューを開く
2. Comm: を選択
3. USB HID に切替 → 自動再起動

コマンド

```
sf sim run
sf sim run -w ringworld
```

操作	説明
スロットル	上昇
ロール / ピッチ	傾き
ヨー	回転

アクロモード（角速度制御）で飛行

注意

終了後は Comm: を ESP-NOW に戻すこと（実機飛行用）

チェックポイント / Checkpoint

確認事項

- ☐ `sf doctor` がエラーなしで通る
- ☐ ビルドが成功し `"Hello StampFly!"` が表示される
- ☐ コントローラのビルド・書き込みが完了した
- ☐ シミュレータが起動し操縦を試した

次のレッスン

Lesson 1: モータ制御 — モータを回してプロペラの動きを確認

Lesson 1

モータ制御

Motor Control

StampFly Workshop

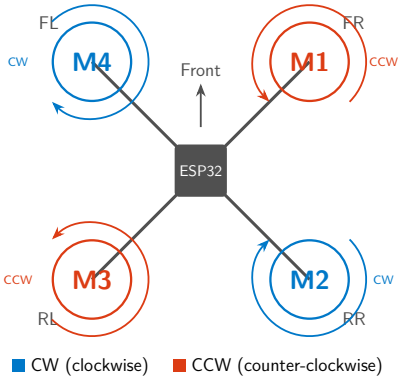
今日のゴール / Today's Goal

目標

モータの番号と配置を理解し、PWM で個別制御する

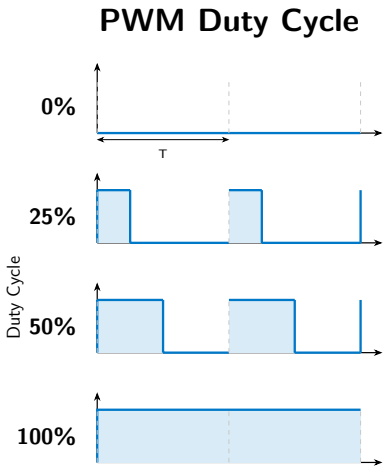
- モータ番号 M1–M4 の位置を覚える
- ARM / DISARM の仕組みを理解する
- PWM デューティ比でモータ回転数を制御

モータ配置 / Motor Layout



- 対角のモータが同じ方向に回転（トルクバランス）
- M1(FR), M3(RL) = CCW / M2(RR), M4(FL) = CW

PWM とは / What is PWM?



- **Pulse Width Modulation**（パルス幅変調）
- **Duty 0%** = 停止
- **Duty 100%** = 最大回転
- API: `motor_set_duty(id, duty)`
  id=1–4, duty=0.0–1.0

ARM / DISARM とは / What is ARM?

安全スイッチ / Safety Switch

- **ARM** = モーター出力を有効化（回転可能になる）
- **DISARM** = モーター出力を無効化（強制停止）
- コードから `arm()` で ARM できる

重要 / Important

`arm()` を呼ばないとモーターは回らない！
`setup()` 内で最初に呼ぶこと

モータ制御 API

関数	説明	引数
`arm()`	モーター出力を有効化	—
`disarm()`	モーター出力を無効化	—
`motor_set_duty(id, duty)`	個別モータ設定	id=1–4, duty=0.0–1.0
`motor_set_all(duty)`	全モーター括	duty=0.0–1.0
`motor_stop_all()`	全モータ停止	—

安全注意 / Safety

duty は 0.15 以下で！ 機体を手で押さえてテスト！

実習: モータを順番に回す / Hands-on

```cpp
#include "workshop_api.hpp"
static uint32_t timer = 0;

void setup() {
    ws::print("Lesson 1: Motor Control");
    ws::arm();  // Enable motor output
}
void loop_400Hz(float dt) {
    timer++;
    // 800 ticks = 2 seconds at 400Hz
    int motor_id = (timer / 800) % 4 + 1;
    ws::motor_stop_all();
    ws::motor_set_duty(motor_id, 0.1f);
}
```

student.cpp

チェックポイント / Checkpoint

確認事項

- ☐ `arm()` なしではモータが回らないことを確認した
- ☐ M1 が FR（右前）のプロペラを回す
- ☐ M1→M2→M3→M4 の順に回転する

次のレッスン

Lesson 2: コントローラ入力 — スティックでモータを操作

Lesson 2
コントローラ入力
Controller Input

StampFly Workshop

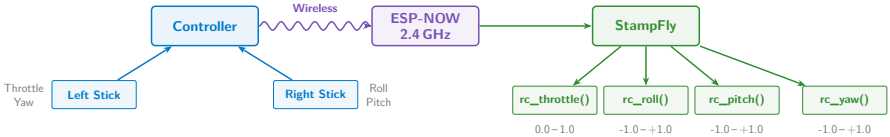
今日のゴール / Today's Goal

目標: スティック値を変数に読み取り、演算でモータを個別制御する

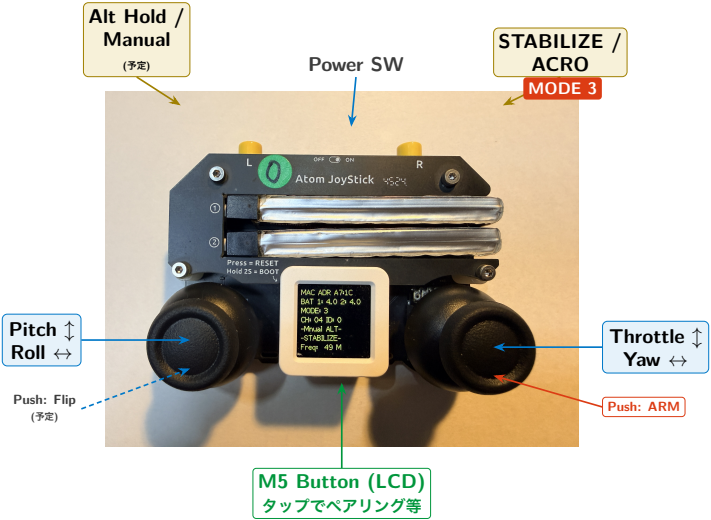
- ESP-NOW 無線通信の仕組みを理解
- チャンネル設定で他の受講生との混信を回避
- 変数と四則演算でスティック → モータ Duty を計算
- オープンループ手動操縦の限界を体感

ESP-NOW 通信フロー / Communication Flow

Controller → ESP-NOW → StampFly API



コントローラ各部 / Controller Layout (MODE 3)



コントローラ API

関数	説明	値域
`set_channel(ch)`	WiFi チャンネル設定	1, 6, 11
`rc_throttle()`	スロットル	0.0 – 1.0
`rc_roll()`	ロール	-1.0 – +1.0
`rc_pitch()`	ピッチ	-1.0 – +1.0
`rc_yaw()`	ヨー	-1.0 – +1.0
`is_armed()`	ARM 状態	true / false
`motor_set_duty(id, v)`	モータ個別制御	id=1–4, v=0.0–1.0

ペアリング手順 / Pairing

Step	操作
1	コントローラの **M5 ボタンを押しながら電源 ON** → LCD に “Pairing mode...” 表示
2	StampFly のボタンを **3 秒長押し** → 青 LED 高速点滅 + ビープ音
3	ペアリング完了 → ビープ音が鳴り、次回から自動接続

注意

- 教室では**一組ずつ**ペアリングする（ブロードキャスト通信のため近くの機体と干渉する可能性）
- うまくいかない場合: 両方を再起動して Step 1 からやり直す

チャンネル設定 / Channel Setting

教室で複数の StampFly を同時に飛ばすと、同じチャンネル同士で**混信**が起きる。チャンネルを分けて回避する。

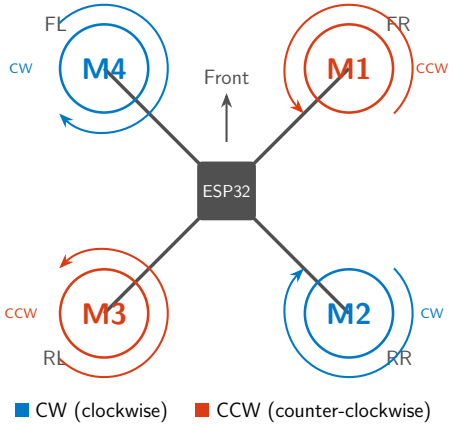
Ch	割り当て例
1	グループ A（緑）
6	グループ B（黄）
11	グループ C（赤）

設定方法

`set_channel(ch)` を `setup()` 内で呼ぶ

機体とコントローラで同じ番号にすること！

モータ配置とミキシング / Motor Layout & Mixing



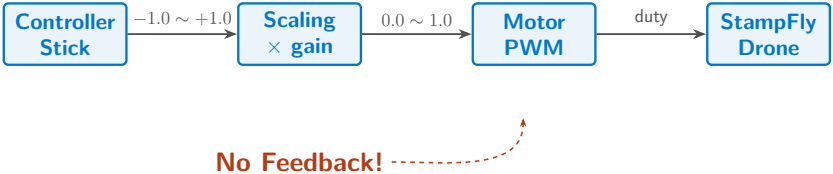
Motor	T	Roll	Pitch	Yaw
M1 FR	+	−	+	+
M2 RR	+	−	−	−
M3 RL	+	+	−	+
M4 FL	+	+	+	−

直感的に理解する

Pitch+ → 前傾 → 前 (M1,M4) 強 + 後 (M2,M3) 弱
Roll+ → 右傾 → 右 (M1,M2) 弱 + 左 (M3,M4) 強

オープンループ制御 / Open-Loop Control

Open-Loop Control



問題点

フィードバックがないため、外乱（風・重心ずれ）に対応できない！
→ Lesson 5–6 で PID 制御を導入して解決

実習: 手動ミキシング (1/2) / Hands-on: Setup

```cpp
#include "workshop_api.hpp"
static uint32_t tick = 0;

void setup() {
    ws::print("L2: Open-Loop Control");
    ws::arm();                  // Enable motor output
    ws::set_channel(1);         // 1, 6, or 11
}
```

student.cpp — setup

チャンネル番号

講師が指定した番号（1 / 6 / 11）に書き換えること！
コントローラも同じチャンネルに設定する

実習: 手動ミキシング (2/2) / Hands-on: Loop

```
void loop_400Hz(float dt) {
    tick++;
    float t = ws::rc_throttle();
    float r = ws::rc_roll()  * 0.3f;
    float p = ws::rc_pitch() * 0.3f;
    float y = ws::rc_yaw()   * 0.3f;
    ws::motor_set_duty(1, t - r + p + y); // FR
    ws::motor_set_duty(2, t - r - p - y); // RR
    ws::motor_set_duty(3, t + r - p + y); // RL
    ws::motor_set_duty(4, t + r + p - y); // FL
    if (tick % 80 == 0)
        ws::print("T=%.2f R=%.2f P=%.2f Y=%.2f",
            t, r, p, y);
}
```

student.cpp — loop

チェックポイント / Checkpoint

確認事項

- ☐ 指定チャンネルで混信なく通信できた
- ☐ コントローラとペアリングできた
- ☐ スロットルで全モータが均等に回る
- ☐ ピッチスティックで前後モータの回転差が出る

次のレッスン

Lesson 3: LED 制御 — システム状態を LED で可視化

Lesson 3: LED 制御

LED Control

StampFly Workshop

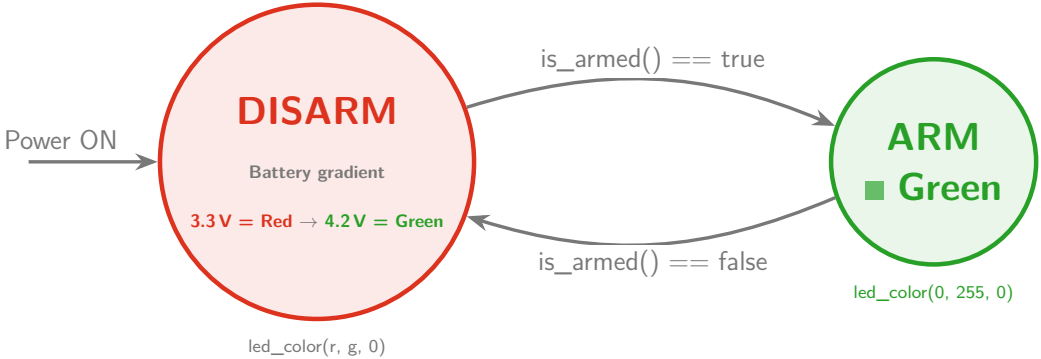
今日のゴール / Today's Goal

目標

LED でシステムの状態（ARM / DISARM）とバッテリー残量を可視化する

- WS2812 アドレサブル LED の制御
- 状態遷移の考え方（ステートマシン）
- バッテリー電圧のモニタリング

LED 状態遷移 / LED State Machine



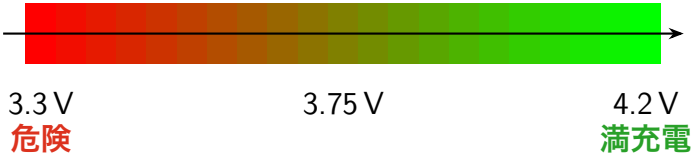
LED 制御 API

関数	説明	引数
`disable_led_task()`	システム LED 更新を無効化	—
`led_color(r, g, b)`	LED 色設定	各 0–255
`is_armed()`	ARM 状態確認	true / false
`battery_voltage()`	バッテリー電圧	3.0–4.2 V

WS2812 LED

- RGB フルカラー（各チャンネル 8bit = 1677 万色）
- 1 本のデータ線で制御（ESP32 RMT ペリフェラル使用）
- `led_color()` は裏表両方の LED を同じ色に設定

バッテリー電圧と色 / Battery Voltage & Color



- LiPo バッテリー: 3.3V（空）〜 4.2V（満充電）
- **3.3V 以下で使用禁止！** バッテリーが損傷します

実習: ARM 状態で LED 色を変える / Hands-on

```cpp
#include "workshop_api.hpp"
void setup() {
    ws::print("Lesson 3: LED Control");
    ws::disable_led_task();  // Student LED control
}
void loop_400Hz(float dt) {
    if (ws::is_armed()) {
        ws::led_color(0, 255, 0);    // Green
    } else {
        // Battery level as color gradient
        float v = ws::battery_voltage();
        float level = (v - 3.3f) / (4.2f - 3.3f);
        if (level < 0) level = 0;
        if (level > 1) level = 1;
        uint8_t r = 255 * (1.0f - level);
        uint8_t g = 255 * level;
        ws::led_color(r, g, 0);
    }
}
```

student.cpp

チェックポイント / Checkpoint

確認事項

- ☐ DISARM 時に赤〜緑のグラデーション表示
- ☐ ARM すると緑に変わる
- ☐ バッテリー残量に応じて色が変化する

次のレッスン

Lesson 4: IMU センサー — 加速度・ジャイロで姿勢を知る

Lesson 4: IMU センサ

IMU Sensor

StampFly Workshop

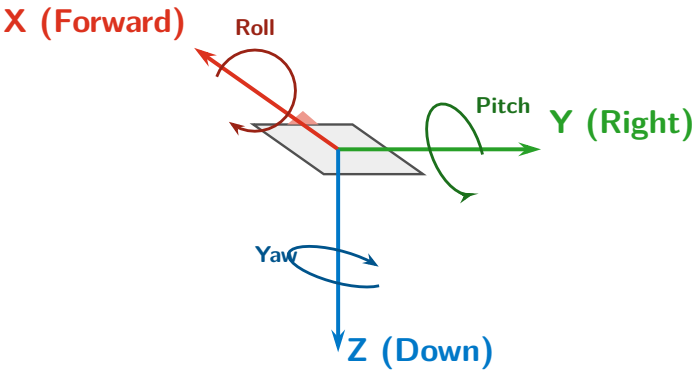
今日のゴール / Today's Goal

目標

IMU（ジャイロ+加速度）データを読み取り、シリアルモニタと WiFi テレメトリで確認する

- NED 座標系（北-東-下）の理解
- ジャイロスコープで角速度を取得
- 加速度センサで並進加速度を取得
- WiFi テレメトリでデータ取得・可視化

NED 座標系 / NED Coordinate System



BMI270 IMU — 6 軸（3 軸ジャイロ + 3 軸加速度）400 Hz　|　回転矢印 = ジャイロ正方向（右手の法則）

IMU API

関数	説明	単位
`gyro_x/y/z()`	角速度 (Roll/Pitch/Yaw)	rad/s
`accel_x/y/z()`	加速度 (X/Y/Z)	m/s^2

静止時の値

ジャイロ ≈ 0、加速度 Z $\approx 9.81\,m/s^2$（重力）

実習: 6 軸読み取り + シリアル表示 / Hands-on

```cpp
#include "workshop_api.hpp"
static uint32_t tick = 0;
void setup() {
    ws::print("Lesson 4: IMU Sensor");
}
void loop_400Hz(float dt) {
    tick++;
    float gx = ws::gyro_x();
    float gy = ws::gyro_y();
    float gz = ws::gyro_z();
    float ax = ws::accel_x();
    float ay = ws::accel_y();
    float az = ws::accel_z();
    // Print every 100ms (40 ticks)
    if (tick % 40 == 0) {
        ws::print("G=(%.3f,%.3f,%.3f) A=(%.2f,%.2f,%.2f)",
                  gx, gy, gz, ax, ay, az);
    }
}
```

student.cpp

WiFi テレメトリ受信 / Receiving WiFi Telemetry

システムが IMU・姿勢・センサデータを自動的に 400 Hz で送信しています

手順

1. シリアルモニタで SSID を確認（`StampFly_XXXX`）
2. PC の WiFi で StampFly に接続（IP: `192.168.4.1`）
3. `sf log wifi -d 30` で 30 秒キャプチャ → CSV 自動保存

出力例

`logs/stampfly_wifi_20260303T143000.csv` に保存されました

`-o name.csv` でファイル名指定、`--no-save` で保存なし（統計のみ）

データの可視化 / Data Visualization

保存した CSV を可視化

```
sf log viz logs/stampfly_wifi_*.csv
```

モード	表示内容
`--mode all`	全パネル（デフォルト）
`--mode sensors`	IMU + 高度センサ
`--mode attitude`	姿勢推定（Roll/Pitch/Yaw）

- `--save plot.png` でファイル保存可能
- StampFly を手で揺らしながらキャプチャし、ジャイロの変化をグラフで確認しよう

チェックポイント / Checkpoint

確認事項

- ☐ シリアルモニタでジャイロ・加速度の値を確認できる
- ☐ 静止時に accel_z ≈ 9.81 を確認
- ☐ `sf log wifi` でセンサデータを受信できる
- ☐ `sf log viz` でデータをグラフ表示できる

次のレッスン

Lesson 5: レート P 制御 — ジャイロフィードバックで初フライト

Lesson 5: レート P 制御 + 初フライト

Rate P-Control + First Flight

StampFly Workshop

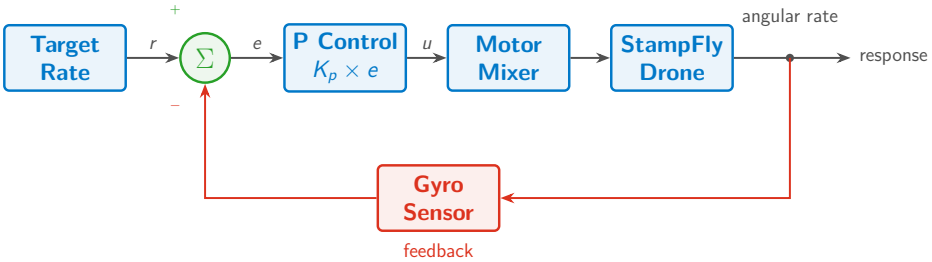
今日のゴール / Today's Goal

目標

比例フィードバック制御を実装し、初の制御飛行を行う

- 閉ループ制御の考え方
- P 制御で角速度を安定化
- ARM/DISARM による安全管理

閉ループ制御 / Closed-Loop Control



制御 API

関数	説明	値域
`gyro_x/y/z()`	ジャイロ角速度	rad/s
`rc_roll()`	ロールスティック	$-1.0 \sim +1.0$
`rc_pitch()`	ピッチスティック	$-1.0 \sim +1.0$
`rc_yaw()`	ヨースティック	$-1.0 \sim +1.0$
`rc_throttle()`	スロットル	$0.0 \sim 1.0$
`motor_mixer(T,R,P,Y)`	モーターミキサー	—
`is_armed()`	ARM 状態確認	true/false

実習: 3軸 P 制御 / Hands-on

```cpp
#include "workshop_api.hpp"
void setup() { ws::print("Lesson 5: Rate P-Control"); }
void loop_400Hz(float dt) {
    if (!ws::is_armed()) {
        ws::motor_stop_all();
        ws::led_color(50, 0, 0);  // Red = DISARM
        return;
    }
    ws::led_color(0, 50, 0);  // Green = ARM
    float Kp_rp = 0.5f, Kp_yaw = 2.0f;
    float rate_max = 1.0f, yaw_max = 5.0f;
    float roll_e  = ws::rc_roll()*rate_max  - ws::gyro_x();
    float pitch_e = ws::rc_pitch()*rate_max - ws::gyro_y();
    float yaw_e   = ws::rc_yaw()*yaw_max   - ws::gyro_z();
    ws::motor_mixer(ws::rc_throttle(),
        Kp_rp*roll_e, Kp_rp*pitch_e, Kp_yaw*yaw_e);
}
```

student.cpp

安全注意 / Safety

フライト前チェック

- □ **保護メガネを着用**
- □ **低スロットルから徐々に上げる**（最初は 30% 以下）
- □ プロペラの回転方向を確認（M1=CW, M2=CCW, M3=CW, M4=CCW）
- □ 異常時はすぐにスロットルを下げて DISARM

チェックポイント / Checkpoint

確認事項

- ☐ ARM 後スロットルを上げて安定ホバリング
- ☐ スティック操作で機体が応答する
- ☐ DISARM で即座にモーターが停止する

次のレッスン

Lesson 6: PID 制御 — I 項・D 項で精度向上